Rx 编译器架构

Semantics、IR 与 Lowering

赵睿城

目录

1 总体架构

1.1 阶段接口

1.2 贯穿示例

1.3 C++ 表示

2 语义分析

2.1 声明与类型

2.2 表达式检查

2.3 地址与控制流

3 IR

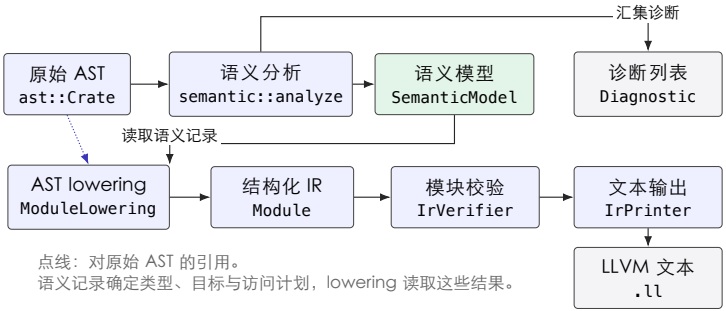
3.1 模型与工具

3.2 Lowering 基础

3.3 表达式与控制流

3.4 聚合与容器

总体数据流与阶段接口



点线：对原始 AST 的引用。
语义记录确定类型、目标与访问计划，lowering 读取这些结果。

跨阶段的数据契约

语义层记录含义，lowering 读取原始 AST 与语义记录。模块、函数与表达式 lowering 共同把这些事实转成 IR。

贯穿示例：函数、局部变量与调用

```
fn add_one(x: i32) -> i32 { x + 1 }
fn main() {
    let n: i32 = 41;
    let y: i32 = add_one(n);
    println_i32(y);
}
```

语义层确定的信息

- 参数与局部绑定的身份和类型。
- 加法与返回表达式的类型。
- 调用目标与参数匹配结果。

需要生成的表示

参数与局部槽、加法指令、函数调用和返回。源码语义下预期输出为 42；后面逐项跟踪其表示。

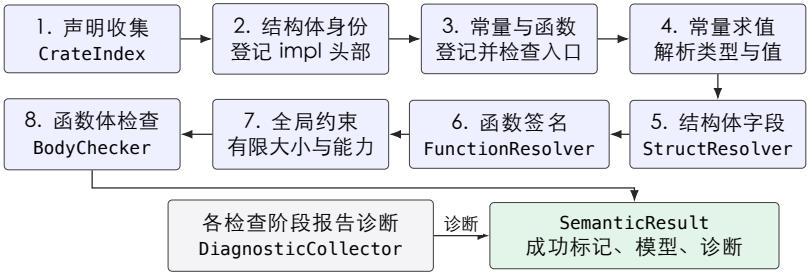
C++ 中的节点、身份与结果表

表示选择	在本仓库中的用途	设计原因
AST 节点与指针	语法结构、源位置，关联语义记录	保留原始结构，复用节点身份
整数 ID 与向量	类型、函数、局部变量、IR 值	表中集中保存对象，引用保持简洁
`std::variant`	类型类别、指令、调用目标	用有限分支明确支持的种类
`map` / `unordered_map`	名字索引、类型驻留、节点旁表	按名字、结构或节点查询已有结果
const 引用与 guards	共享模型与作用域状态	明确读写责任，退出时恢复状态

身份与生命周期

ID 分属不同空间。AST 指针键必须引用仍然有效的原始节点。

语义分析的执行顺序

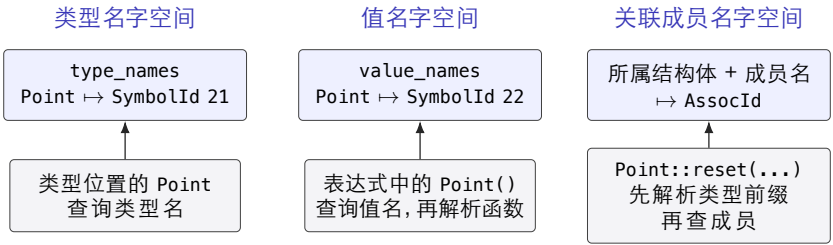


阶段顺序对应当前 `analyze`。关键错误检查点提前结束后续阶段，并保留已收集的诊断。

先准备事实，再检查函数体

先建立名字、类型与签名，函数体才能解析前向引用。关键阶段失败时提前结束，并返回已收集的诊断。

CrateIndex 与名字空间



同名结构体与函数可位于不同名字空间；图中 ID 仅作示意。
局部值先由 ScopeManager 查找最近绑定；字段通过所属结构体的字段表解析。

声明收集的输出

CrateIndex 保存符号及全局名字索引，关联项再按所属结构体查询。名字先解析为身份，后续检查使用该身份。

TypeContext：类型驻留与相等

类型类别	TyInfo 保存什么	相等时关注什么
基本整数与 bool	`PrimaryTyKind`	四种整数保持各自身份
命名结构体	声明的 `SymbolId`	同一声明，不比较字段是否碰巧相同
引用	目标 TyId 与可变性	目标和 mut 必须匹配
数组、Box、Vec	元素 TyId；数组另存长度	结构化参数与长度
unit / never / error	各自独立的 variant 分支	三者承担不同检查含义

驻留的作用

结构化类型集中驻留，表达式与签名通过 TyId 引用已有结果。

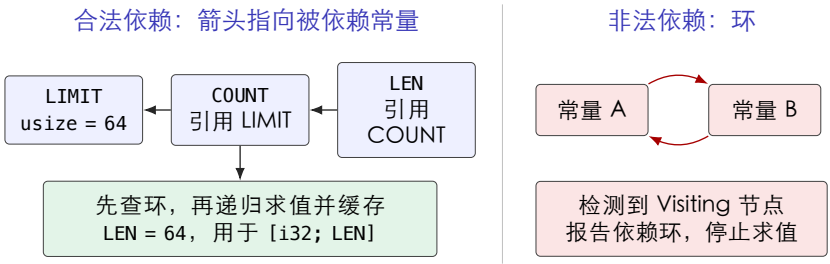
TypeResolver：类型写法到语义类型

输入或上下文	解析动作	形成的结果
AST 的类型节点	识别名称、引用、数组等形式	一个语义 TyId
命名类型与 `Self`	查询类型名；使用当前结构体上下文	结构体声明身份
引用、Box、Vec	递归解析内部类型，检查参数形式	结构化类型节点
数组长度	交给 ConstEvaluator，要求 usize 常量	元素类型与确定长度
非法名称或参数	写入诊断，返回错误类型	后续阶段可识别失败

类型解析与表达式推导分工

类型写法交给 TypeResolver，表达式类型交给 BodyChecker。

ConstEvaluator：常量依赖与求值



ConstEvaluator 处理受限常量上下文；普通运行时表达式的常量折叠属于另一项可选优化。

受限常量上下文

const 初始化值、数组类型长度和数组重复长度使用规定的常量形式。先确认依赖无环，再求值并缓存。

StructResolver 与 ImplResolver

StructResolver

- 先为所有结构体登记身份。
- 再解析字段类型与字段顺序。
- 保存字段名索引和请求的 derives。

ImplResolver

- 确认 impl 的目标结构体。
- 登记关联函数与关联常量。
- 按所属类型检查成员名冲突。

为后续查询准备结果

字段访问查询 StructInfo，关联项查询 AssocInfo。声明身份先建立，因此类型可以引用稍后才展开的声明。

FunctionResolver：函数身份与签名

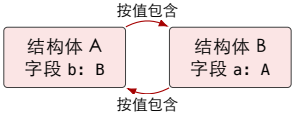
准备的事实	记录位置	后续用途
函数身份与类别	`FunctionInfo`	区分顶层、关联函数与内建函数
参数名、类型与 mut	`ParameterInfo`	建立参数局部变量，检查实参
返回类型与 receiver 模式	`FunctionSignature`	检查返回、方法选择与调整
AST 声明关联	声明指针与 FunctionId 映射	定位函数体，传递给 lowering
源入口约束	`checkEntryPoint`	要求合法的源 main

签名先于函数体

调用检查只需先取得完整签名，无需等待被调用函数的函数体检查完成，因此可以处理前向调用与递归。

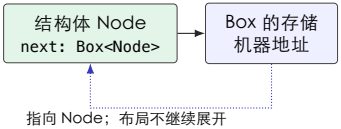
全局约束：有限大小与内建能力

按值包含：展开后必须有限



A 的大小依赖 B，B 又依赖 A
LayoutChecker 拒绝这种环

间接引用：对象内只存地址



数组元素仍按值展开；引用、Box、Vec 不产生该类按值递归边。
LayoutChecker 检查有限性；TargetLayout 才计算具体大小、偏移与对齐。

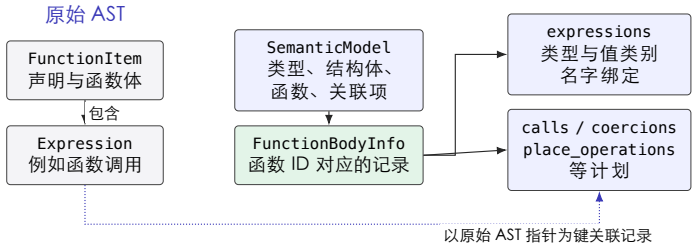
LayoutChecker

检查按值包含的依赖是否有环，保证对象大小可以有限展开。

DeriveChecker

根据字段与请求的 derives，检查 Copy、Clone、PartialEq、Eq 能力。

SemanticModel 与函数体语义旁表



lowering 同时读取原始 AST 和语义旁表。AST 节点地址必须保持有效，不能换一棵重建的树来查表。

分析结果需要留下来

SemanticModel 保存全局事实；FunctionBodyInfo 保存局部绑定、表达式信息与计划。后续阶段沿用原始节点查表。

BodyChecker、检查上下文与作用域

每个函数的检查上下文

- 当前 FunctionInfo 与局部变量表。
- ScopeManager 与当前循环栈。
- 可达状态、错误标记和语义记录。

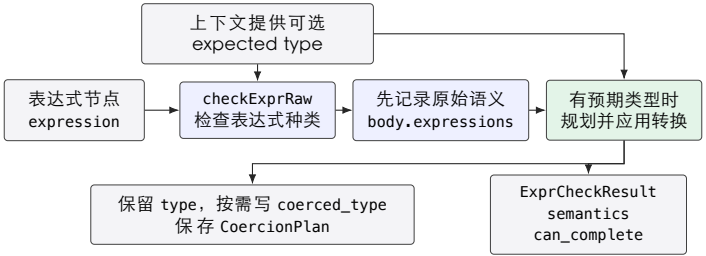
作用域中的绑定

- 参数先登记为 LocalId。
- let 的初始化先于新名字绑定。
- 查找从内层作用域向外进行。

遮蔽不会改变旧绑定身份

同名的新 let 得到新的 LocalId。ScopeGuard 负责进出作用域，降低提前返回时忘记恢复状态的风险。

表达式检查：原始结果与转换



无 expected 时返回原始结果；类型错误进入诊断。
预期类型的传播由 Rx 规则限定；类型与可完成性分开记录。

ExprSemantics 的主要维度

type 与 coerced_type 描述类型，category 与 access 描述值或位置及访问属性，绑定字段描述解析到了哪个对象。

OperatorChecker 与操作数计划

运算类别	需要判断什么	留给 lowering 的事实
算术与位运算	操作数类型是否满足该类规则	运算类型与结果类型
移位	左值整数种类与右侧规则	有符号或无符号右移语义
相等与有序比较	源类型、能力、引用读取方式	OperandPlan 与 bool 结果
一元运算	允许的标量及读取方式	UnaryPlan
二元运算	两侧的读取方式与类型	BinaryPlan

一个 IR i32 承载多种源语义

i32、u32、isize、usize 在源层仍不同。除法、比较与右移的指令选择需要保留源类型信息。

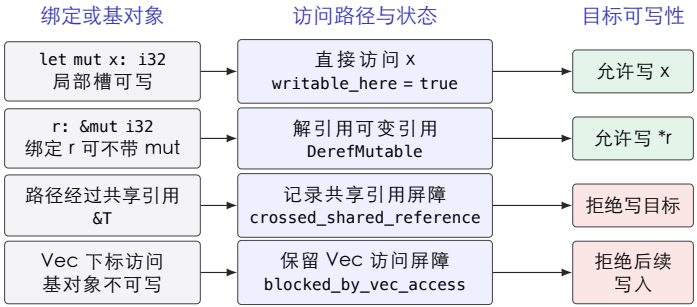
CoercionChecker、预期类型与 LUB

场景	处理方式	保存结果
已知目标类型	对单个表达式尝试转换	CoercionPlan
多个结果分支	选择共同目标并覆盖结果	LubPlan 与各处转换计划
类型保持一致	保留类型，或进行重借用	记录具体转换种类
引用调整	内建解引用后再次借用	PlaceStep 与访问属性
never 到目标类型	路径终止，不生成值	保留控制流终止事实

预期类型不等于任意隐式转换

整数间转换需要显式 as。预期类型只按 Rx 规定传播，不能任意穿过运算符或借用来重新推断类型。

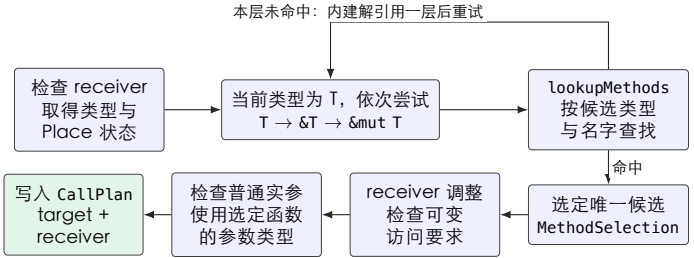
PlaceChecker：位置与可写性



先判断访问是否合法

PlaceChecker 记录解引用、字段、下标和临时物化步骤。绑定的 mut 与最终位置是否可写，需要沿整条访问路径判断。

普通调用、方法选择与 CallPlan



先选方法，再校验 receiver 与实参；不能因后续校验失败而回退到另一个候选。
内建解引用支持引用与 Box；Vec 不参与内建解引用。候选歧义或最终无匹配进入诊断路径。

调用结果也是计划

普通调用保存 target；方法调用还保存 receiver 的解引用与借用调整。选定目标后检查实参，lowering 再消费这些记录。

控制流语义：类型与可完成性

维度或结构	保存什么	解决的问题
ExprSemantics.type	值的原始语义类型	该表达式产生哪种值
can_complete	能否沿当前表达式正常完成	后续路径是否仍可继续
LoopFrame / LoopId	循环类别、break 位置及结果类型	break / continue 指向哪里
分支结果与 LUB	各结果位置和共同目标	表达式结果如何统一
ReachabilityGuard 等	进入嵌套检查前的状态	返回时恢复可达性与循环上下文

检查与执行路径分开理解

return、break 等会终止当前路径；不可达位置仍有需要检查的静态规则，具体排除项以 Rx 规范为准。

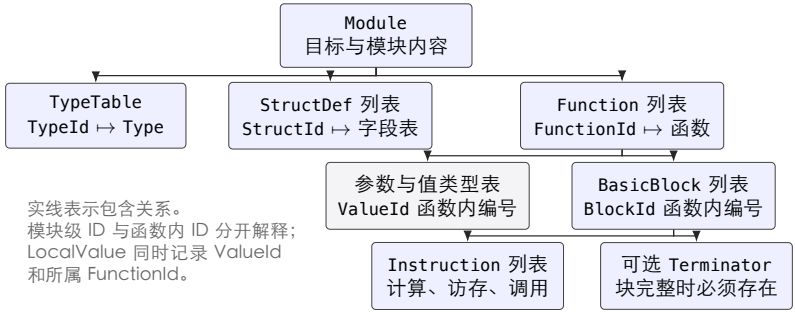
语义分析留下的结果与诊断

贯穿示例中的对象	语义记录	后续消费者
参数 x、绑定 n 和 y	局部表与参数、let 映射	参数绑定与局部存储
对局部变量的读取	LocalId 与 TyId	找局部槽并读取
参数加一的表达式	BinaryPlan 与结果类型	选择并构造加法
add_one 与 I/O 调用	CallPlan 的目标与结果类型	映射到 IR 函数并调用
名字或类型检查失败	诊断级别、源位置与消息	汇集为 SemanticResult

错误结果不能继续生成 IR

success 为 false 时保留诊断供用户定位。旁表中的 ErrorTy 或 Recovery 是恢复机制，不能作为正确 lowering 的输入。

结构化 IR 的对象组成



先在内存中建立结构

本仓库用自己的 C++ 数据结构组织 LLVM 风格 IR。Module 是整体结果，Printer 最后才将其序列化为文本。

TypeTable、Operand 与 IR 身份

组成	表示内容	身份与归属
TypeTable	void、整数、ptr、数组、命名结构体	TypeId 选择一种 IR 类型
StructDef	名称与可选字段定义	StructId 定位结构体定义
Function	签名、参数、值表、块	FunctionId 在模块中定位函数
Operand	TypeId + 立即数、局部值或空指针	局部值记录 ValueId 与函数归属
BasicBlock	指令序列与终结指令	BlockId 在函数内定位块

语义类型与 IR 类型分开

语义 TyId 保存语言含义，IR TypeId 保存目标表示。即使数值相同，也必须经过 TypeLowering 的映射。

指令、结果值与终结指令

指令与结果

- Binary、ICmp、load 等产生结果值。
- store 和 void call 没有结果值。
- Operand 存输入，指令对象存操作。

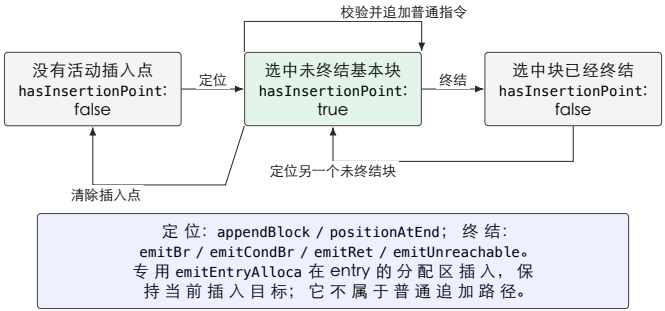
基本块的结束方式

- Br：无条件跳转。
- CondBr：按 bool 条件分支。
- Ret / Unreachable：终止路径。

```
%v1 = add i32 %v0, 1
store i32 %v1, ptr %slot, align 4
ret i32 %v1
```

LLVM 指令片段示意：局部值必须在所属函数中合法定义，完整块还必须有 terminator。

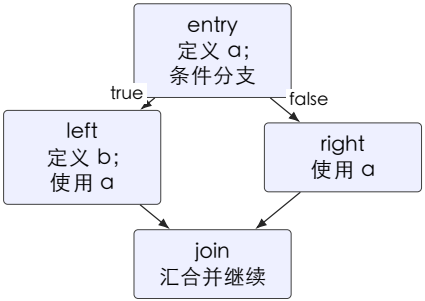
IrBuilder：构造过程与插入点



把构造规则放在集中接口

emitter 检查操作数类型、函数归属、签名、对齐和插入状态。专用 entry alloca 接口与普通块尾追加分别处理。

IrVerifier：CFG 与定义支配使用



join 使用 a：合法
所有到达 join 的
路径都经过定义

join 直接使用 b：非法
right 路径没有
经过 b 的定义

`IrVerifier`
建 CFG、算可达
性与支配关系

局部合法不等于全局合法

Builder 检查一次构造是否合理；Verifier 检查定义完整性、块终结、CFG 和跨块使用等整体约束。

IrPrinter：结构化对象到 LLVM 文本

组件	主要工作	输出
IrBuilder	创建对象，检查本次操作的局部条件	修改后的 Module
IrVerifier	检查模块和函数的整体约束	带位置字段的 IrError 列表
IrPrinter	输出目标、类型、函数、指令与块结束方式	LLVM 文本流
LLVM 工具	解析文本并检查 LLVM 的规则	外部解析或验证结果

定义与引用保持一致，正确处理类型、符号转义、立即数与 terminator。

文本输出职责

Printer 按 Module 记录输出符号、类型和指令，控制流结构与类型映射由 lowering 层决定。

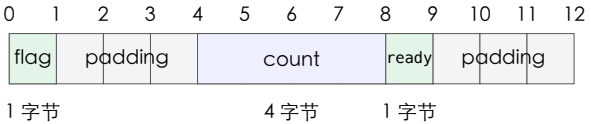
TypeLowering：存储类型与计算类型

语义类型	storage / value 类型	需要保留的区别
四种 32 位整数	i32 / i32	符号与运算规则仍来自语义类型
bool	i8 / i1	读取时截断，存储时零扩展
引用与 Box	ptr / ptr	目标类型由语义记录继续提供
数组、结构体	对应聚合类型 / 聚合类型	字段与元素采用存储表示
Vec	描述符结构体 / 同一结构体	字段类型为 ptr、i32、i32
unit / never	不提供普通存储或计算值	unit 返回 void；never 路径不产生值

TargetLayout：大小、偏移与对齐

```
Sample { flag: bool, count: i32, ready: bool }
```

字段按声明顺序存放；下方数字为字节偏移



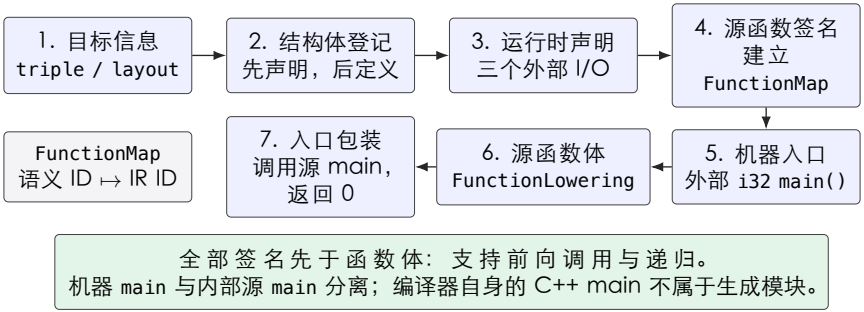

```
TypeLayout
size = 12
alignment = 4
offsets=[0,4,8]
```

`bool`：存储使用 i8，计算值使用 i1；padding 不对应语义字段。
TargetLayout 使用 RV32 目标参数，结构体总大小向最大字段对齐取整。

布局由目标决定

TypeLayout 保存 size、alignment、field_offsets 和 array_stride。目标为 RV32，大小与对齐按目标参数计算。

ModuleLowering：登记、映射与入口



先登记，后填充函数体

函数调用通过语义 FunctionId 到 IR FunctionId 的映射完成。机器 main 是单独的包装函数，调用内部源 main 后返回 0。

FunctionLowering：局部槽与函数结果

函数内的工作

- 建 entry，绑定参数到局部槽。
- 按原顺序生成语句与块尾表达式。
- 读取结果状态，再决定函数返回。

```
struct BodyResult {
    bool reaches_end;
    std::optional<Operand> value;
};
```

状态	含义
true + 有值	正常完成，得到 Operand
true + 无值	正常完成，结果是 unit
false + 无值	路径终止，后续不再插入

存储与值各有身份

按 LocalId 保存局部槽；entry 分配存储，执行点初始化。

表达式 lowering：读取计划并生成操作

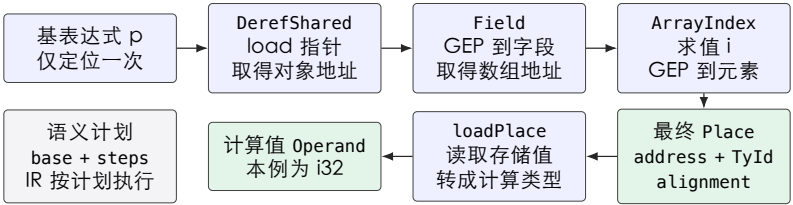
步骤	使用的信息	生成行为
查询语义记录	expressions 与 effectiveType	确认当前表达式的预期结果
分派原始表达式	AST 类别与运算、调用等 Plan	调用具体 lowering 路径
按源顺序处理子表达式	子表达式 BodyResult	子路径终止则停止后续生成
执行已记录的转换	CoercionPlan 与地址步骤	保持 unit、never 和借用语义
检查返回约定	值类型、是否有值、插入状态	结果与语义记录一致

标量运算仍依赖源语义

有符号与无符号除法、比较、右移分别选择。移位量屏蔽到低 5 位，普通整数加减乘按 32 位模式处理。

地址 lowering：执行 PlacePlan

读取 p.items[i]： p: &Packet，字段 items: [i32; 4]

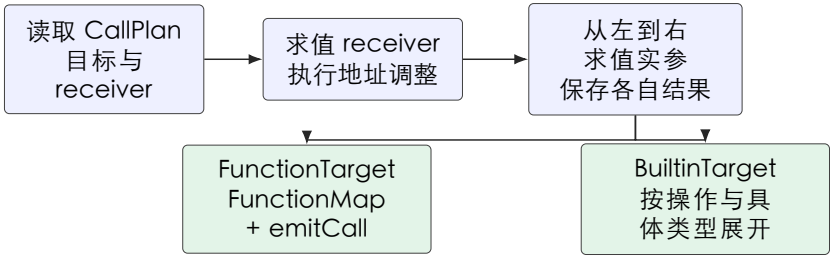


GEP 只计算地址；解引用存储在槽中的指针需要 load。
本例经过共享引用，只展示读取；storePlace 用于语义层已允许写入的目标。

基表达式只求值一次

PlaceCursor 在值与地址状态间推进，applyPlaceSteps 执行既定路径。loadPlace / storePlace 集中处理存储与计算表示转换。

调用 lowering：消费 CallPlan



普通调用省略 receiver 步骤。任何子表达式终止路径，外层调用随即停止生成。

调用目标在语义层确定

FunctionTarget 通过 FunctionMap 找到 IR 函数，BuiltinTarget 按操作种类与 owner_type 生成对应行为。

方法 lowering：receiver 成为显式参数

ReceiverPlan 信息	生成动作	传给被调方
expression	求值基表达式一次	保留原始值或位置
steps	解引用、投影与物化	得到调整后的游标
Value	读取最终值	按值 receiver 参数
BorrowShared / BorrowMutable	取得最终地址	引用 receiver 参数
parameter_type	选择参数的 IR 表示	与函数签名一致

方法调用的执行顺序

点调用执行 ReceiverPlan，随后从左到右求值实参。

转换 lowering：原始结果与有效结果

转换形式	执行方式	结果约定
Identity	生成原始表达式结果	保留其值或 unit
NeverToAny	生成原始表达式的终止路径	reaches_end = false
BorrowShared / BorrowMutable	从 Raw 结果执行 place_steps	返回最终位置的地址
整数 as 整数	保留 32 位模式	后续按目标符号性解释
bool as 整数	将 i1 零扩展为 i32	false 为 0，true 为 1

每个表达式只执行一次

lowerExpr 选择一种转换路径，lowerExprRaw 负责原始表达式。借用路径复用 Raw 结果，避免再次求值。

操作数读取：求值之后再执行运算

OperandReadKind	求值阶段保存什么	操作阶段读取什么
Value	标量计算值	直接使用该值
SharedReferent	一层共享引用的值	读取其目标标量
OrderedReferent	带引用深度的引用值	沿计划读取最终有序标量
EqualityBorrow	用于相等比较的借用结果	按源类型递归比较内容

左、右操作数的求值先完成

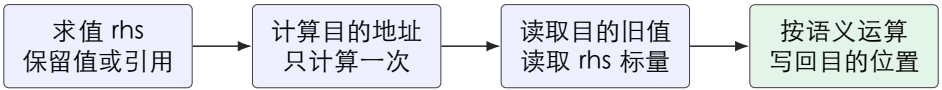
先求值左侧，再求值右侧，随后读取引用目标并运算。保留 source_type 与 operation_type，选择正确指令。

赋值 lowering：右侧先行与一次寻址

普通赋值：`place = value`



复合赋值：`place += rhs`

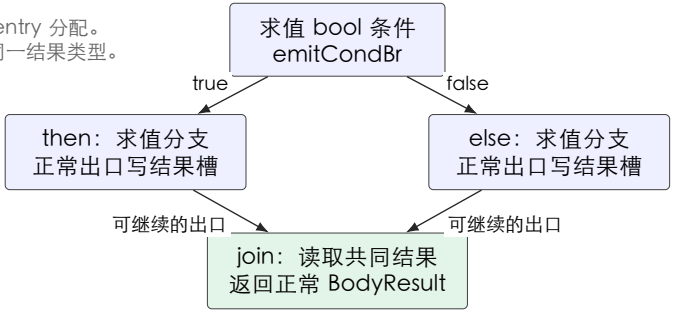


地址与读写分开生成

复合赋值在右侧和目的地址求值后读取旧值。引用 rhs 也在此时读取目标标量，两种赋值均返回 unit。

if lowering：分支结果与汇合

结果槽在 entry 分配。
分支共享同一结果类型。

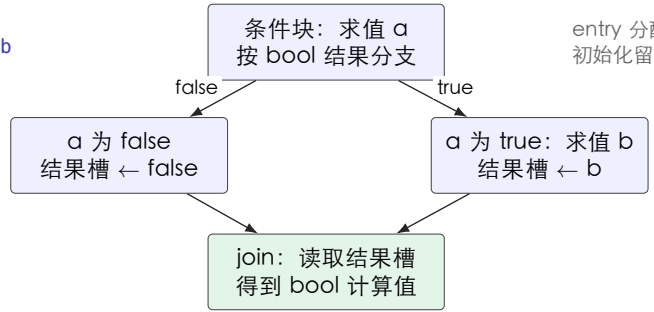


汇合只接收能继续的路径

结果槽使用语义层选定的共同类型。unit 不分配结果槽，终止分支不写槽，也不跳到 join。

短路 lowering：由 CFG 决定是否求值

a && b

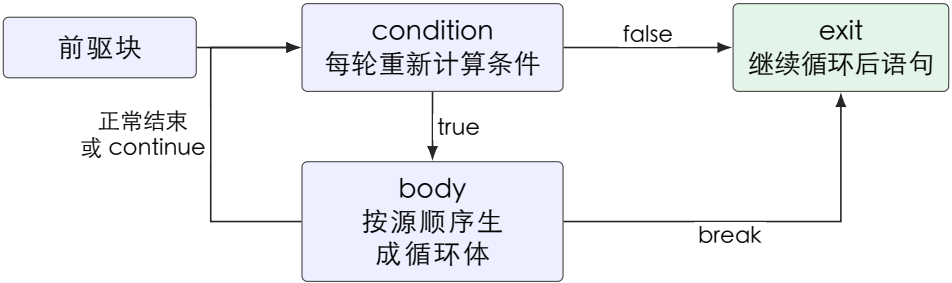


entry 分配结果槽，
初始化留在分支中。

运行路径体现短路语义

a && b 在 a 为 true 时执行 b 分支，a || b 在 a 为 false 时执行 b 分支。

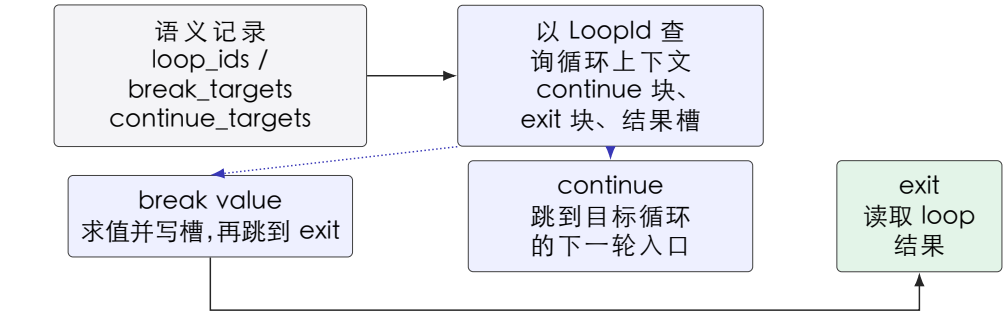
while lowering：条件块与回边



continue 回到条件求值处

循环体正常结束也跳回 condition，break 跳到 exit。while 的结果是 unit，循环体尾值不作为循环结果。

loop lowering：LoopId 与结果槽



嵌套循环通过身份定位目标

continue 跳到 loop 的循环体入口，带值 break 写入目标循环的结果槽。正常退出后，从该槽取得循环表达式结果。

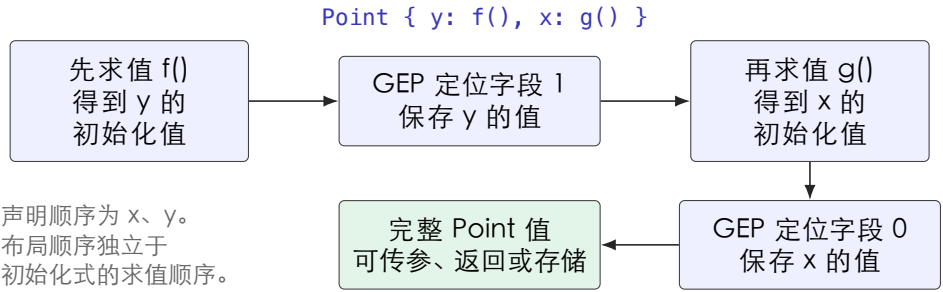
return 与 BodyResult 的逐层传播

表达式或语句	生成的控制流	向外传递的结果
return value	求值返回值，再 emitRet	false + 无值
break / continue	写入必要结果，再 emitBr	false + 无值
普通块正常结束	保留尾表达式的结果	true + 有值或无值
函数体正常结束	finishReturn 处理尾值	生成函数返回
内部表达式终止	停止外层剩余步骤	继续传递 false + 无值

terminator 与插入状态保持一致

终止一条路径后，Builder 不再向该块追加指令。块表达式把状态向外返回，函数返回由函数层收口。

结构体 lowering：求值顺序与字段序号



按源码求值，按声明布局存放

先建立聚合结果的存储，再逐个生成初始化式。字段序号决定 GEP 位置，类型决定存储表示与对齐。

数组 lowering：列表与重复构造

源形式	求值与构造	使用的类型信息
列表 [a, b, c]	从左到右求值，写入对应元素	确定的元素类型与长度
重复 [value; N]	value 求值一次，重复保存结果	N 来自常量求值
元素位置	用 GEP 定位数组元素	元素 storage 类型与 stride
数组整体值	保留固定长度的聚合表示	参数和返回不衰退成指针
len 方法	求值 receiver，返回数组长度	生成 usize 常量

重复构造保留一次求值

N 大于 1 时元素类型需要 Copy，所有元素复用同一份求值结果。

聚合值传递：复制、移动与局部存储

使用位置	lowering 处理	必须保留的行为
let 与整体赋值	保存完整数组或结构体值	后续修改只影响目的值
按值参数	参数拥有独立的值语义	修改参数不改写调用方值
返回聚合值	返回整体值或等价的结果存储	调用方可继续持有结果
Copy 读取	复制字段或整体表示	源值继续可用
move 读取	转移值及容器所有权	不隐式创建第二份拥有者

地址传递也要保留按值语义

可以选择聚合 Operand 或结果存储作为内部表示，参数、赋值和返回必须采用一致的传递约定。

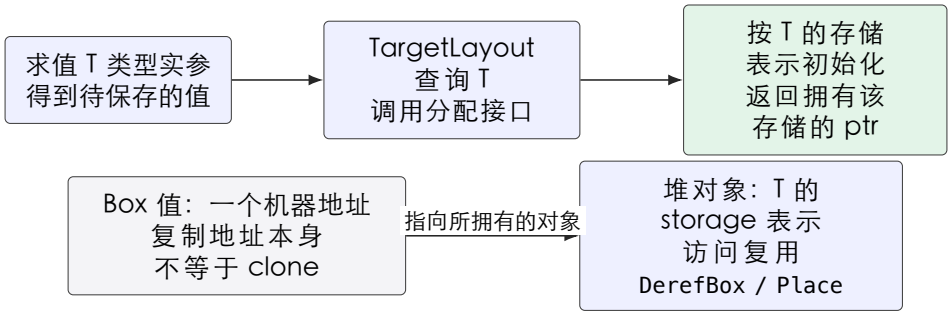
Clone 与相等比较：按类型生成行为

类型	Clone	PartialEq
标量与共享引用	复制标量或引用值	标量按值，引用比较目标
结构体与数组	递归克隆字段或元素	逐字段或逐元素比较
Box	分配新对象并克隆内容	比较所存内容
Vec	按顺序克隆到独立缓冲	先比较长度，再比较元素

类型专用 helper 可复用递归逻辑

按 Tyld 与操作种类缓存 helper，先登记签名再生成函数体。比较跳过 padding，容器容量和分配地址不参与相等性。

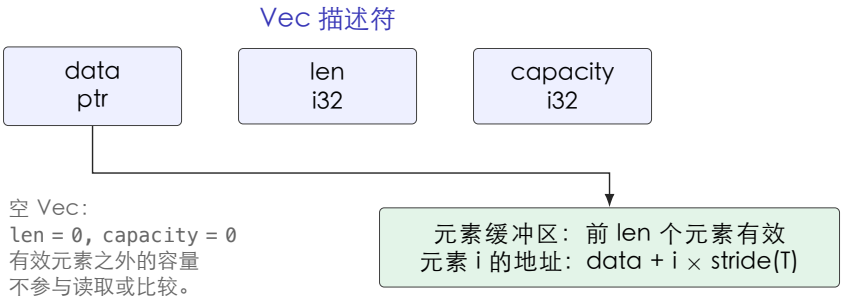
Box lowering：类型布局与堆对象



分配接口处理未类型化存储

`__rx_alloc(size, align)` 取得字节区域。编译器计算 T 的大小与对齐，并生成初始化、访问和 clone 行为。

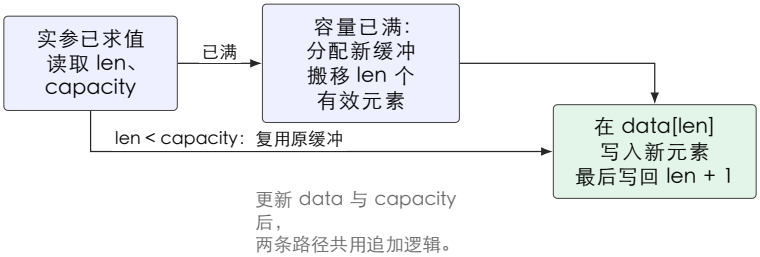
Vec lowering：描述符与元素访问



元素地址沿用统一的 Place 表示

len 读取长度，is_empty 比较长度与 0，索引从 data 和元素 stride 计算地址，再按 T 的表示读写。

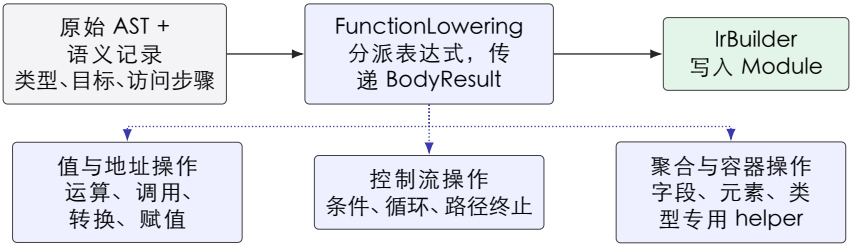
Vec 更新：扩容、追加与移除



remove(index) 的数据顺序

保存被移除的元素，向左搬移后续元素，再减小 len，最后返回保存的值。重叠的搬移区间使用 memmove 或等价循环。

Lowering 组件的协作



共享 TypeLowering、TargetLayout、FunctionMap 与 Place 访问接口。模块完成后交给 Verifier 与 Printer。

语义计划贯穿每一层

调用目标、转换、地址路径和循环身份来自同一份语义记录，各类表达式通过 BodyResult 组合成函数体。